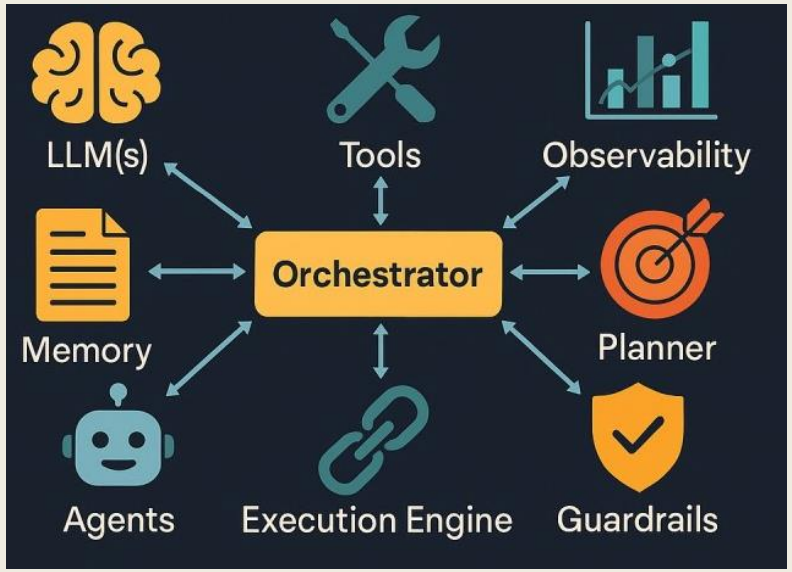
Building Multi-Agent System

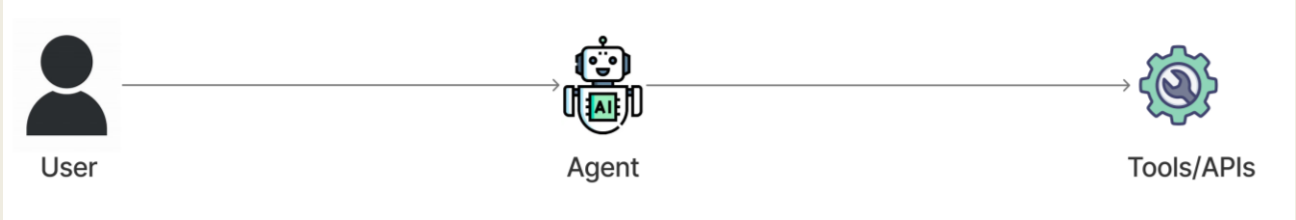
- **What is an Orchestrator Agent?**
  - A specialized agent that:
    - Interprets user goals
    - Chooses the right agent or tool
    - Routes tasks
    - Monitors execution
    - Merges results
    - Manages retries and fallbacks
  - Orchestrator = **the brain** of your multi-agent system.

- **Responsibilities of the Orchestrator:**
  - Task Decomposition – Breaks down the user request into smaller tasks.
  - Skill/Agent Selection – Choose which agent/tool is appropriate
  - Execution Routing – Creates the workflow of agent
  - State Management - Tracks progress, Input/Outputs, Context across agents
  - Conflict Resolution – Handles inconsistent result, multi agent responses, disagreements
  - Safety & Guardrails – Ensure compliance with policies, PII handling, content guardrails
  - Final Output Assembly – Combines all agent outputs into a final response.



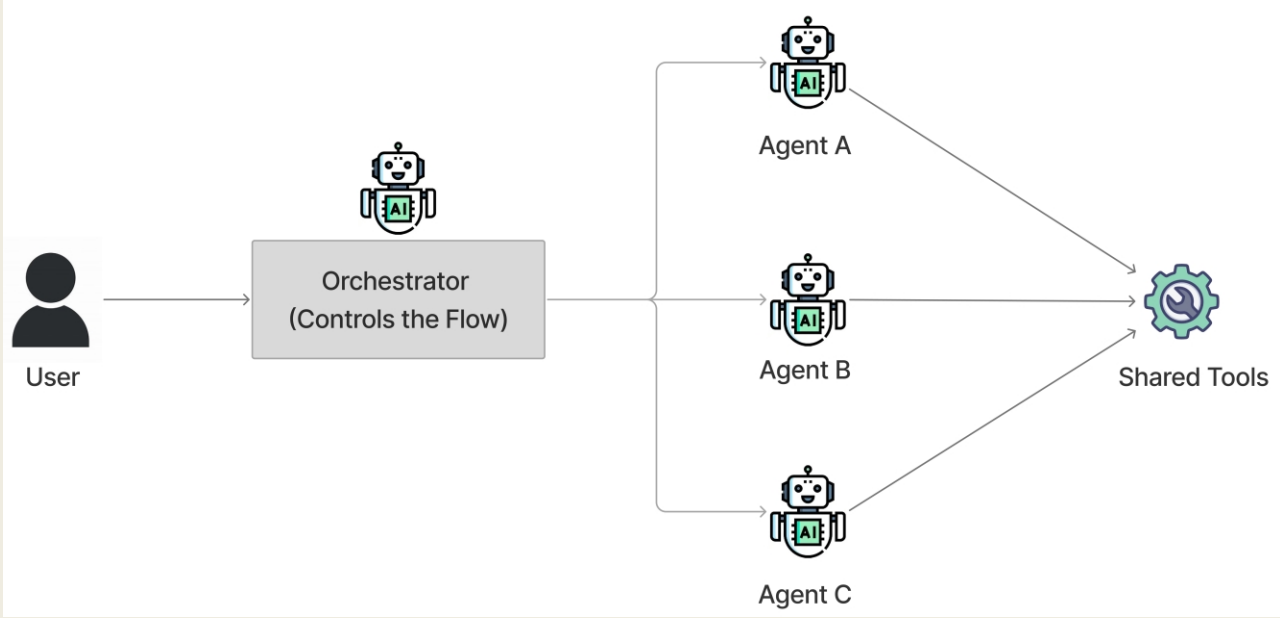
Building Multi-Agent System

- **Why Do we need an Orchestrator?**
  - **Without Orchestrator:**
    - Agents become independent silos
    - No unified decision-making
    - No multi-step workflows
    - No safety consistency
    - No coordination



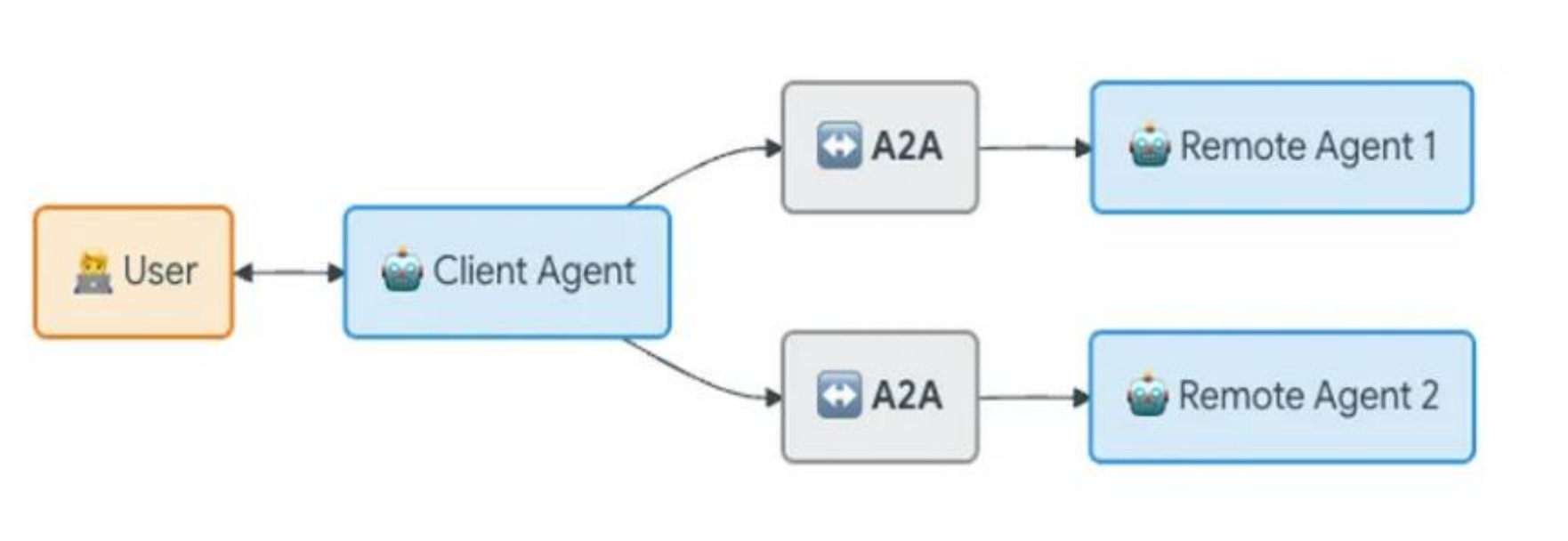
  - **With Orchestrator:**
    - Clean architecture
    - Each agent becomes reusable
    - Agents can be treated like tools

    by the orchestrator
    - Predictable workflows
    - Easy debugging and observability



Building Multi-Agent System

- **What is A2A Protocol?**
  - Designed to provide seamless communication and collaboration between AI agents.
  - Built for cross-platform enterprise-scale collaboration.
  - Agents on different devices, clouds, or organizations can communicate securely.
  - Designed for distributed workflows where multiple agents (LLM, database, planners) collaborate horizontally.
  - For example, it can allow gen AI agents, built on diverse frameworks or built by different companies running on separate servers (remotely accessible), to communicate and collaborate effectively — as agents, not just as tools.



Building Multi-Agent System

- **Realizing That Agents Can Also Be Tools**
  - **Agents as Tools**
    - An architectural pattern in AI systems where specialized AI agents are wrapped as callable functions (tools) that can be used by other agents.
    - This creates a hierarchical structure where:
      - **A primary "orchestrator" agent** handles user interaction and determines which specialized agent to call
      - **Specialized "tool agents"** perform domain-specific tasks when called by the orchestrator
    - This approach mimics human team dynamics, where a manager coordinates specialists, each bringing unique expertise to solve complex problems. Rather than a single agent trying to handle everything, tasks are delegated to the most appropriate specialized agent.
  - **Key Benefits and Core Principles**
    - **Separation of concerns**: Each agent has a focused area of responsibility, making the system easier to understand and maintain
    - **Hierarchical delegation**: The orchestrator decides which specialist to invoke, creating a clear chain of command
    - **Modular architecture**: Specialists can be added, removed, or modified independently without affecting the entire system
    - **Improved performance**: Each agent can have tailored system prompts and tools optimized for its specific task

Building Multi-Agent System

- **Realizing That Agents Can Also Be Tools**
  - **Technical Implementation**
    - **Step 1 : Define the Orchestrator (Main Agent)**
      - Design the primary agent whose role is to understand complex user goals, plan a solution path, and delegate tasks. This agent needs strong reasoning capabilities.
      - Configure the orchestrator LLM for Tool Use
        - Use a large language model (LLM) that supports function calling. Provide the LLM with the *definitions* of the functions/tools.
    - **Step 2: Identify Specialized Tasks & Create Tool Agents**
      - Break down potential tasks into narrow, specific functions. Each function that can be encapsulated becomes a candidate for a dedicated "Tool Agent" (which might be a simple function, an API call, or a fine-tuned mini-agent).
      - Define Tools as Functions:
        - Wrap the logic of specialized agents into standard function definitions. These functions should have clear names, input parameters, and expected output types.

Building Multi-Agent System

- **Realizing That Agents Can Also Be Tools**
  - **Technical Implementation**
    - **Step 2: Identify Specialized Tasks & Create Tool Agents**
      - *Example*: For an e-commerce assistant: a "Search Inventory" agent, an "Apply Discount Code" agent, and a "Track Order Status" agent.
    - **Step 3: Map Dependencies:**
      - Determine which tool agents to orchestrator can call and in what sequence
    - **Step 4: Implement the Interaction Loop**
      - **User Input**: The user asks, "Where is my order #123?"
      - **Orchestrator Reasons**: The LLM determines that to answer this question, it needs to call the track_order function.
      - **Function Call Generation**: The LLM generates a structured call: track_order(order_id="123").
      - **Execution**: Application code executes this function.
      - **Observation Return**: The function's result ("Order #123 is currently in transit.") is returned to the LLM as an observation.
      - **Final Response Generation**: The LLM uses the observation to formulate the final, natural language answer for the user: "Your order #123 is currently in transit."

Building Multi-Agent System

- **How A2A works?**
  - **Capability discovery:** Agents advertise their capabilities using an ‘Agent Card’ in JSON format, enabling other agents to identify the best agent for a task.
  - **Task management:** Communication is oriented towards task completion, with a defined lifecycle that can be complete immediately or over time.
  - **Collaboration:** Agents can send messages to communicate context, replies, artifacts, or user instructions.
  - **User experience:** Messages include ‘parts’ with specified content types, allowing agents to negotiate the correct format and UI capabilities.



- **Why A2A?**
  - **Break down silos:** Connect agents across different ecosystems.
  - **Enable complex collaboration:** Allow specialized agents to work together on tasks that a single agent cannot handle alone.
  - **Promote open standards:** Foster a community-driven approach to agent communication, encouraging innovation and broad adoption.
  - **Preserve opacity:** Allow agents to collaborate without needing to share internal memory, proprietary logic, or specific tool implementations, enhancing security and protecting intellectual property

Building Multi-Agent System

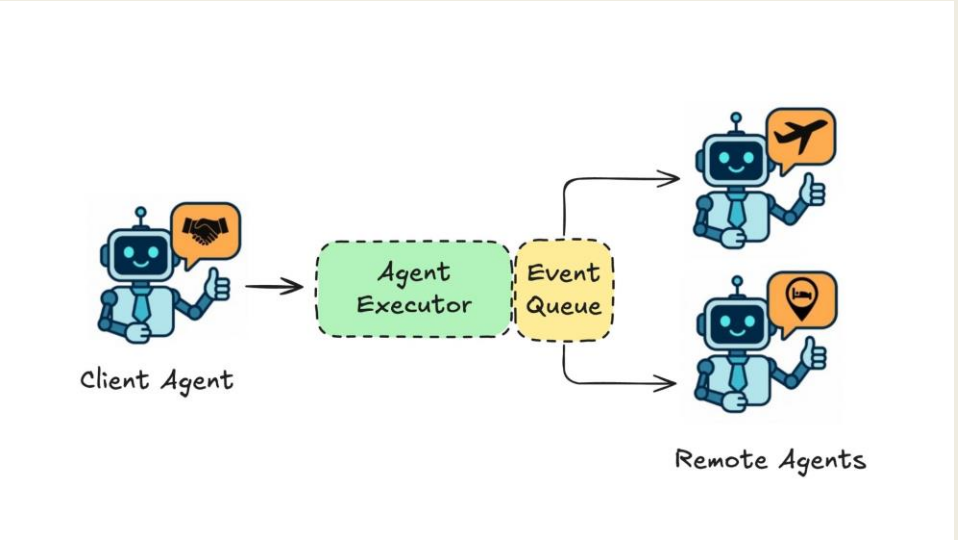
- **Core Components of A2A:**
  - **A2A client:** An application or agent that initiates requests to an A2A server on behalf of a user or another system.
  - **A2A server (remote agent):** It receives requests from clients, processes tasks, and returns results or status updates.
  - **Agent card:** A JSON metadata document published by an A2A Server, describing its identity, capabilities, skills, service endpoint, and authentication requirements.
  - **Message:** A communication turn between a client and a remote agent, having a role (“user” or “agent”) and containing one or more ***parts.***
  - **Task:** The fundamental unit of work managed by A2A, identified by a unique ID. Tasks are stateful and progress through a defined lifecycle.
  - **Part:** The smallest unit of content within a Message or Artifact (e.g., TextPart, FilePart, DataPart).
  - **Artifact:** An output (e.g., a document, image, structured data) generated by the agent as a result of a task, composed of parts.
  - **Streaming (SSE):** Real-time, incremental updates for tasks (status changes, artifact chunks) delivered via Server-Sent Events.

Building Multi-Agent System

- **Task vs Messages in A2A**
  - **Messages:**
    - Used for initial greetings and capability discovery
    - To return response for quick internal actions
    - General conversation that doesn't express a clear intent to perform an action
  - **Task**
    - A user expresses a clear intent to achieve a goal
    - The goal completion can require the agent to collaborate over multiple turns
    - The operation is long-running and required asynchronous tracking
  - **Components:**
    - **Task ID:**
      - A unique ID for the **goal the agent is working on.**
      - Helps track its progress (submitted → working → completed/failed)
    - **Message ID:**
      - A unique ID for **each individual message** inside a task
      - Messages are exchanged within the context of a specific task and carry instructions, questions, answers, or status
    - **Context ID:**
      - A unique ID that **groups related tasks together**
      - Represents a whole session or bigger interaction
      - Example: If a user asks an agent to “Plan my trip”, this may include separate tasks but linked by common context ID →
        - Task 1: Book flight
        - Task 2: Reserve hotel
        - Task 3: Find activities
        - Context ID = ctx-98ab7e10

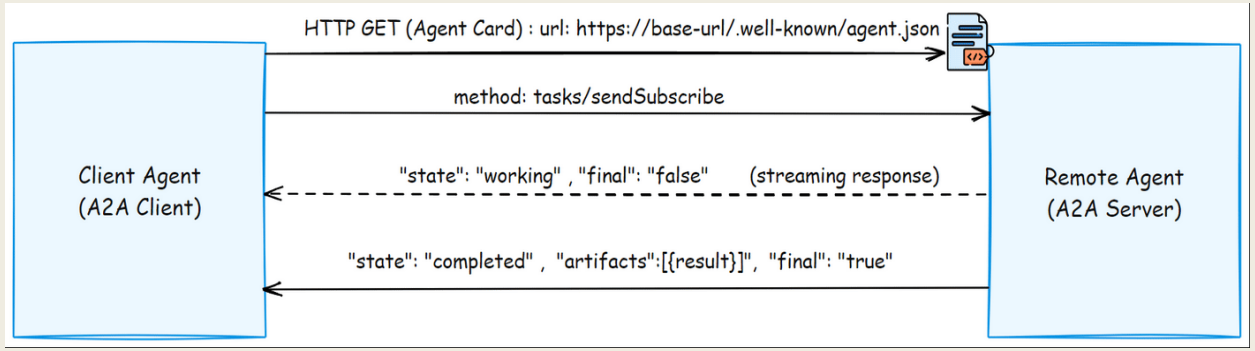
Building Multi-Agent System

- **Agent Executor**
  - A standard for enabling communication and collaboration between AI agents.
  - A lightweight abstraction layer that provides a standard way to **execute** and **cancel** tasks.
  - **Functionality**
    - Handles A2A requests:
      - Receives requests from A2A clients or other agents.
      - Translates these into actions the underlying AI agent can process.
    - Manages task lifecycle:
      - Creates tasks and manages tasks.
      - Monitors responses from underlying LLM/agent
      - Maintains correct task state (created -> running -> completed)
    - Process agent response:
      - Takes raw agent output.
      - Formats it into standardized A2A events/responses to send back to clients or other agents.
    - Enables cancellation:
      - Implements **cancelTask()** to gracefully stop ongoing work.
      - Publishes task canceled events (when supported).
    - Integration with AI Agent Core:
      - Acts as a bridge between high-level requests and the internal logic of AI agents.
      - Ensures consistent behavior across different agent types.



Building Multi-Agent System

- **Agent to Agent Server**
  - Exposes the HTTP(s) endpoints that clients connect
  - An agent server can support streaming or push notification(to client) modes.
  - The connection rules between agent server and client are predefined, establishing a set of specific endpoints.
  - This information is present in the agent's card under capabilities.
  - the Agent server acts as an intermediary, connecting the external world to the agent executor.
- **Agent to Agent Client**
  - When an application or agent seeks to connect with another agent via A2A, it must implement an A2A client that specifies its communication preferences.
  - This includes determining whether the client will:
    - Poll for updates.
    - Configure push notifications from the A2A server.
    - Use HTTPX client for connections.



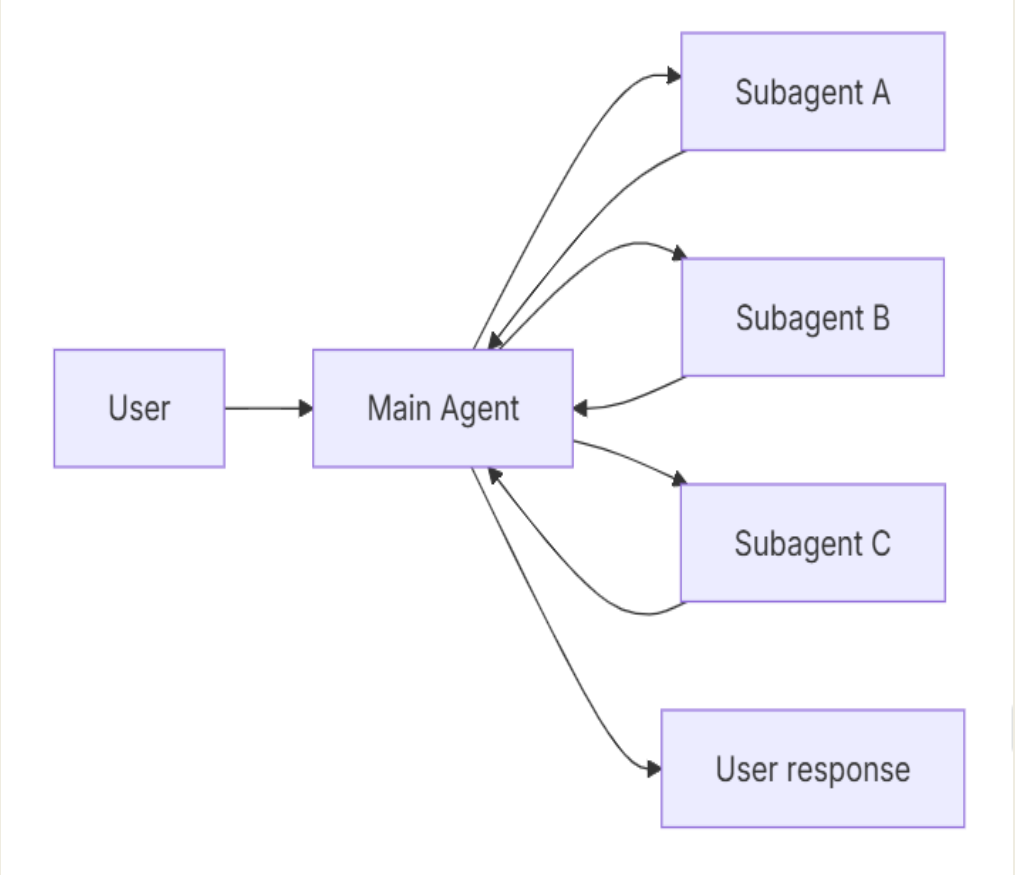
Building Multi-Agent System

Multi Agent(Subagents) - Implementation in LangChain

What it is:
- Main agent coordinates subagents by calling them as tools
- Subagents are stateless—don't remember past interactions
- Main agent maintains all conversation memory
- Provides context isolation for each subagent

When to use:
- Multiple distinct domains (calendar, email, CRM, database)
- Subagents don't need to converse directly with users
- Want centralized workflow control
- Need context isolation between domains

Main implementations:
1. **Basic implementation** - Wrap subagent as tool
2. **Sync vs async execution** - Blocking vs background
3. **Tool patterns** - Tool per agent vs single dispatch tool
4. **Context engineering** - Control context flow (specs, inputs, outputs)



Building Multi-Agent System

Basic Implementation

What it is:

- Wrap subagent as tool that main agent can call
- Main agent invokes subagent when needed
- Subagent returns result to main agent

When to use:

- Have specialized agents for different domains
- Need simple coordination between agents
- Want main agent to control workflow

Business Scenario:

- Personal assistant with specialized agents for calendar, email, and research
- User asks: "Schedule a meeting with John and research his company"
- Main agent calls calendar subagent to schedule, then research subagent for company info

```python
from langchain.tools import tool
from langchain.agents import create_agent

# Create a subagent
subagent = create_agent(model="anthropic:claude-sonnet-4-20250514", tools=[...])

# Wrap it as a tool
@tool("research", description="Research a topic and return findings")
def call_research_agent(query: str):
    result = subagent.invoke({"messages": [{"role": "user", "content": query}]})
    return result["messages"][-1].content

# Main agent with subagent as a tool
main_agent = create_agent(model="anthropic:claude-sonnet-4-20250514", tools=[call_research_agent])
```

How it works:

- **create_agent()** - Create subagent with its own tools
- **@tool decorator** - Wrap subagent as tool for main agent
- **subagent.invoke()** - Call subagent with query
- **return result["messages"][-1].content** - Return final message
- **main_agent with tools** - Main agent can call subagent tool

Building Multi-Agent System

Sync vs Async Execution

What it is:

- **Sync** - Main agent waits for subagent to complete
- **Async** - Main agent continues while subagent runs in background

Mode	Main agent behavior	Best for
Sync	Waits for subagent	Main agent needs result to continue
Async	Continues while subagent runs	Independent tasks, user shouldn't wait

When to use sync:

- Main agent needs subagent's result to formulate response
- Tasks have order dependencies (fetch data → analyze → respond)
- Subagent failures should block main agent's response

When to use async:

- Subagent work is independent of main conversation
- Users should continue chatting while work happens
- Want to run multiple independent tasks in parallel

Building Multi-Agent System

Tool Per Agent

What it is:

- Each subagent wrapped as separate tool
- Fine-grained control over each subagent

When to use:

- Need customization for each subagent
- Different input/output handling per subagent
- Small number of subagents with distinct behaviors
- Want explicit control over each subagent's invocation

Business Scenario:

- Customer support system with billing, technical, and sales subagents
- User: "I was charged twice and my app keeps crashing"
- Main agent calls billing subagent for refund, then technical subagent for troubleshooting

```python
# Create subagents
billing_agent = create_agent(model="...", tools=[refund_tool, check_invoice_tool])
technical_agent = create_agent(model="...", tools=[debug_tool, check_logs_tool])

# Wrap each as separate tool
@tool("billing_agent", description="Handle billing and payment issues")
def call_billing_agent(query: str):
    result = billing_agent.invoke({"messages": [{"role": "user", "content": query}]})
    return result["messages"][-1].content

@tool("technical_agent", description="Handle technical issues and bugs")
def call_technical_agent(query: str):
    result = technical_agent.invoke({"messages": [{"role": "user", "content": query}]})
    return result["messages"][-1].content

# Main agent with multiple subagent tools
main_agent = create_agent(model="...", tools=[call_billing_agent, call_technical_agent])
```

How it works:

- **Separate @tool for each** - Each subagent wrapped individually
- **Custom descriptions** - Main agent knows when to use each
- **Multiple tools** - Main agent has one tool per subagent

Building Multi-Agent System

Single Dispatch Tool

What it is:

- Single tool invokes any subagent by name
- Convention-based approach
- Scalable for many agents

When to use:

- Distribute agent development across multiple teams
- Need to isolate complex tasks into separate context windows
- Scalable way to add new agents without modifying coordinator
- Many agents or dynamic agent registry (10+)
- Prefer convention over customization

Business Scenario:

- Enterprise system with 20+ specialized agents (HR, Finance, Legal, IT, Sales, etc.)
- Different teams develop and maintain their own agents
- User: "What's my vacation balance and submit expense report"
- Main agent dispatches to HR agent, then Finance agent

```python
@tool
def task(agent_name: str, description: str) -> str:
    """Launch an ephemeral subagent for a task."""
    subagent = get_agent_from_registry(agent_name)
    result = subagent.invoke({"messages": [{"role": "user", "content": description}]})
    return result["messages"][-1].content

main_agent = create_agent(model="...", tools=[task])
```

How it works:

- **Single task tool** - One tool for all subagents
- **agent_name parameter** - Select which subagent
- **get_agent_from_registry()** - Dynamically load subagent
- **Convention over customization** - Simpler composition